

CSP初赛集训
第8课
算法概念

2023.07

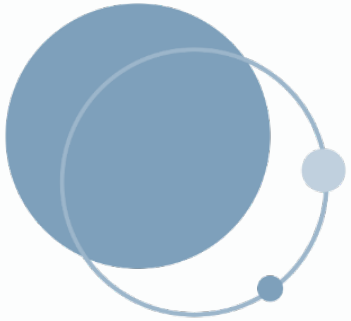
目录/CONTENTS

01 算法概念

02 伪代码

03 主定理

04



01 算法概念



算法概念

算法（Algorithm）是对特定问题求解步骤的一种描述，它是指令的有限序列，其中每一条指令表示一个或多个操作。算法是解决问题的操作步骤。

算法概念

算法的特征：

- **有穷性**：每一条指令的执行次数及时间是有限的。
- **确定性**：每一条指令的含义明确，无二义性。
- **可行性**：算法中描述的操作都是可以通过有限次的基本运算来实现的。
- **输入**：一个算法应该具有0个或多个输入
- **输出**：一个算法应该具有1个或多个输出

算法思想

- 递推
- 递归
- 枚举
- 搜索
- 分治
- 贪心
- 模拟
- 动态规划

算法 习题

【NOIP2000 普及组】算法是指（　　）.

A．为解决问题而编制的计算机程序

B．为解决问题而采取的方法与步骤

C．为解决问题而需要采用的计算机语言

D．为解决问题而采用的计算方法

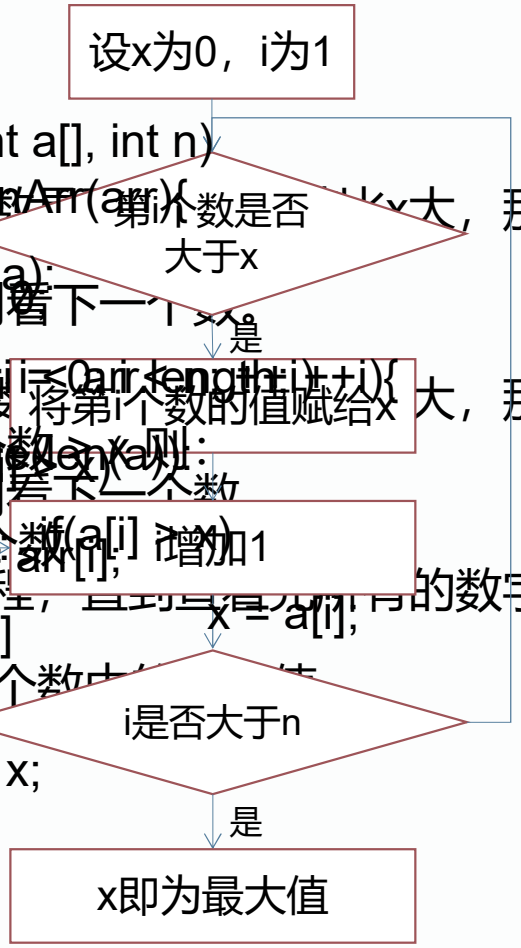
正确答案：B

算法的描述

- 自然语言
- 图形工具
- 伪代码
- 程序代码

求n个自然数中的最大值 的算法：

1. 设x为0，i为1
2. 查看第i个数是否大于x，如果大于x，那么将第i个数的值赋给x，否则看下一个数。
3. 查看第i个数是否大于x，如果大于x，那么将第i个数保存为x，否则看下一个数
4. 重复这一过程，直到查看完所有的数字。
5. 数字x即为n个数中的最大值




```
//C++
int maxInArr(int a[], int n)
{
    int x = 0;
    for(int i=0; i<n; i++){
        if(a[i] > x)
            x = a[i];
    }
    return x;
}
```

算法 习题

【NOIP2001 普及组】下面关于算法的错误说法是( )

A. 算法必须有输出

B. 算法必须在计算机上用某种语言实现

C. 算法不一定有输入

D. 算法必须在有限步执行后能结束

正确答案：B

如何评价算法优劣

- 正确性
- 时间复杂度
- 空间复杂度
- 可读性
- 可移植性
- 健壮性

算法 习题

【NOIP2000 普及组】在下列关于计算机算法的说法中，不正确的是（ ）。

A. 一个正确的算法至少要有一个输入

B. 算法的改进，在很大程度上推动了计算机科学与技术的进步

C. 判断一个算法的好坏的主要标准是算法的时间复杂性与空间复杂性

D. 目前仍然存在许多涉及到国计民生的重大课题，还没有找到能够在计算机上实施的有效算法

正确答案：A

指数与对数

- $N = a^x$　a:底数 x:指数 N:幂

N为a的x次幂

- $x = \log_a N$　a:底数 N:真数 x:对数

x是以a为底N的对数

- $\log_a b$：a的几次方是b

对数公式

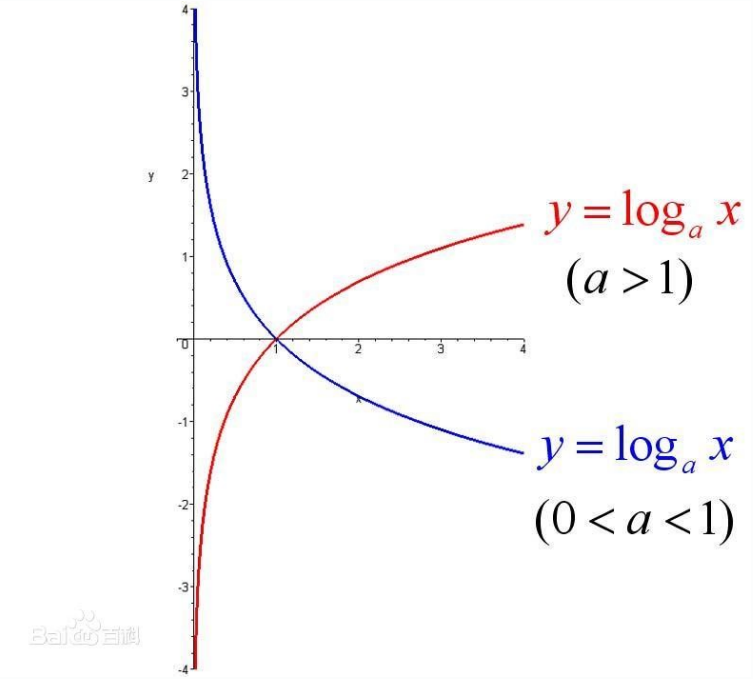
$$\log_a (M \cdot N) = \log_a M + \log_a N$$

$$\log_a \frac{M}{N} = \log_a M - \log_a N$$

$$\log_a b = \frac{\log_c b}{\log_c a}$$

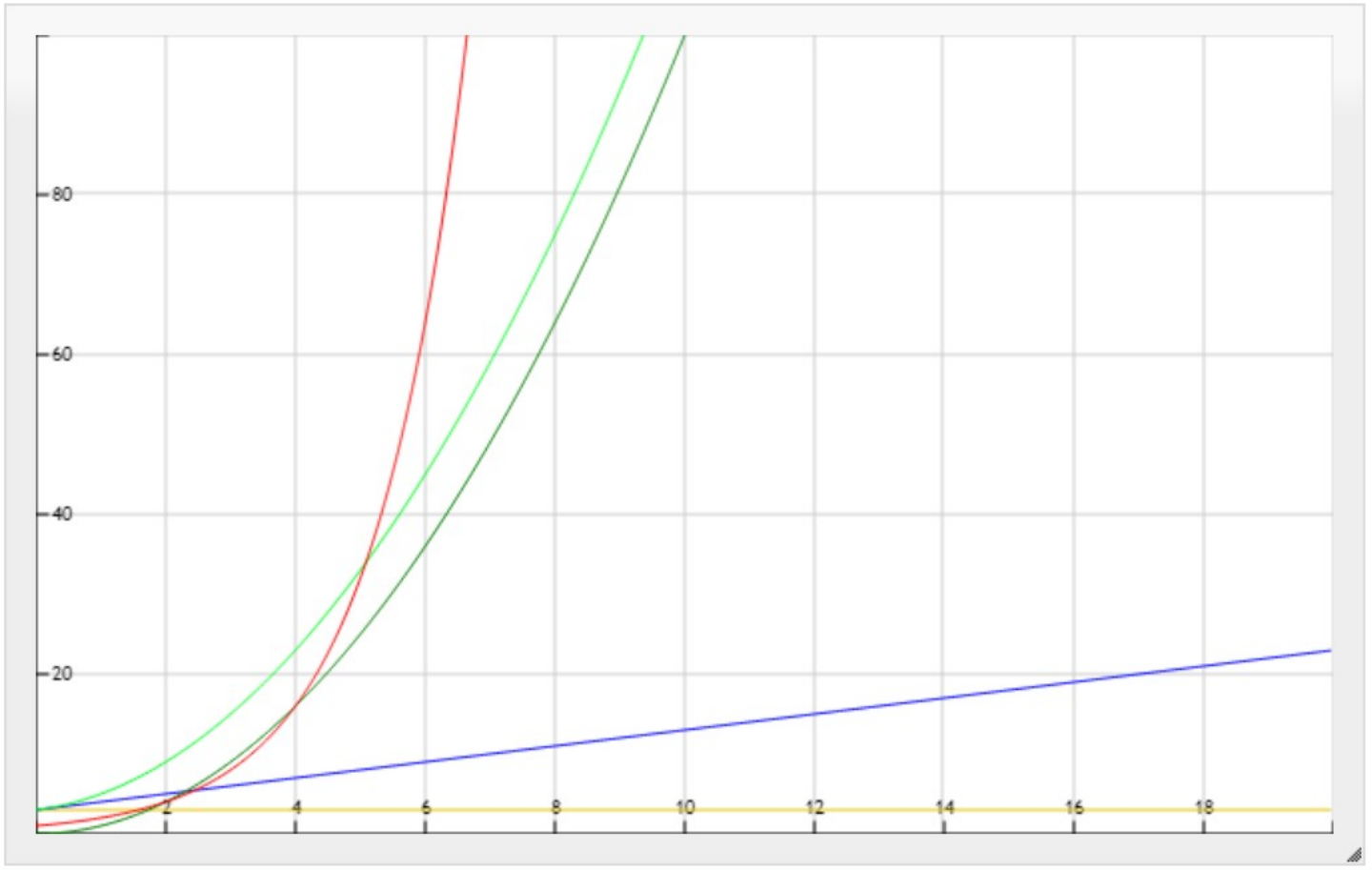
$$\log_a M^n = n\log_a M$$

$$a^{\log_a N} = N$$




```
//求log_ab
cout << log(b) / log(a);
```

函数增长的速度


80
60
40
20
2
4
6
8
10
12
14
16
18
普通函数 y(x)
3
普通函数 y(x)
x+3
普通函数 y(x)
x^2+x+3
普通函数 y(x)
x^2
普通函数 y(x)
2^x
普通函数

大O表示法

O(f(n))为函数f(n)的大O表示法，表示函数f(n)的增长速度。

O()可以视为一种运算符，其运算过程为：

f(n)只保留最高次项，略去其他项，略去与最高次项相乘的常数。

例：

$f(n) = 5$	$O(f(n)) = O(1)$
$f(n) = 2n^2+n+1$	$O(f(n))= O(n^2)$
$f(n) = 3*2^n+n^3+2$	$O(f(n))= O(2^n)$

常用的函数阶

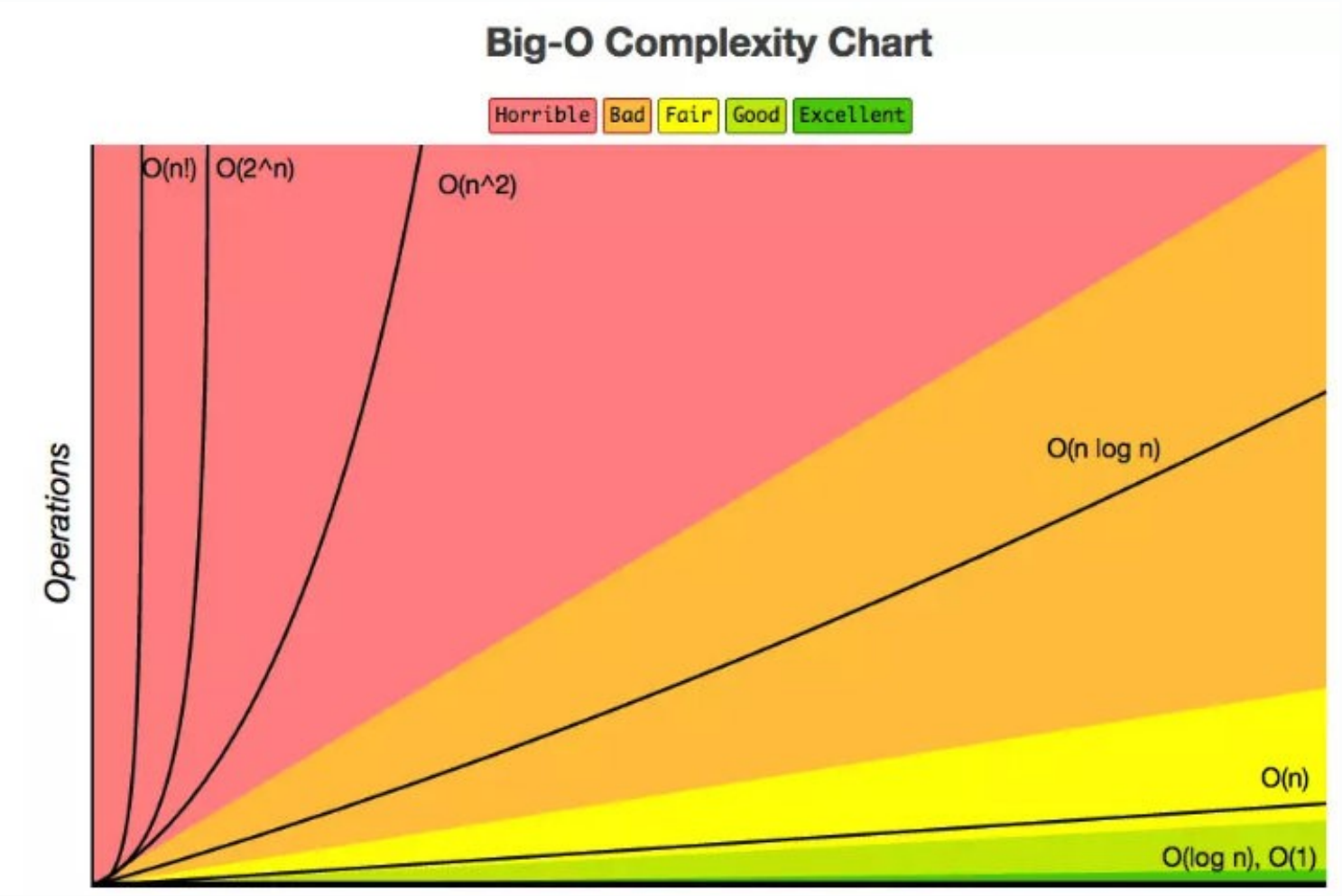
低阶函数增长得慢，高阶函数增长得快。

低

高

符号	名称
O(1)	常数阶
O(log n)	对数阶
O(n)	线性阶
O(n log n)	线性对数阶
$O(n^2)$	平方阶
$O(n^k)$	多项式阶
$O(c^n)$	指数阶
O(n!)	阶乘阶

常用的函数阶


Big-O Complexity Chart
Horrible
Bad
Fair
Good
Excellent
O(n!)
O(2^n)
O(n^2)
O(n log n)
O(n)
O(log n), O(1)
Operations

练习

求以下多项式的大O表示法

1. $-n^2+8n+5$
2. $3n+2n\log_{10}n$
3. $2n+\log_2 n+9$
4. $n!+3^n$
5. $n(n+1)/2$

算法复杂度



时间复杂度要求

如果不满足，会返回
TLE：Time Limit Exceeded（运行超时）

空间复杂度要求

如果不满足，会返回
MLE：Memory Limit Exceeded（内存超限）

算法复杂度

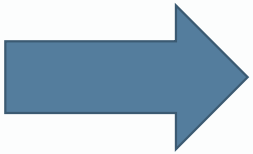
时间复杂度：是指执行这个算法所需要的计算工作量。即算法执行的基本运算次数。

时间频度：一个算法中的语句执行次数，记为T(n)，其中n是问题规模。

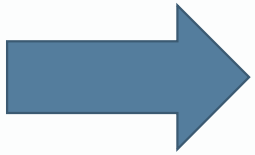
算法的**时间复杂度**，就是时间频度T(n)的大O表示法，即O(T(n))。

算法的时间复杂度O(n)

```
temp = a;
a = b;
b = temp;
```

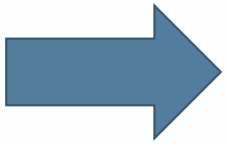


T(n) = 3

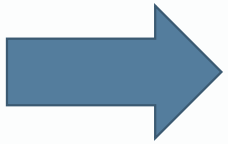


O(1)

```
int s = 0;
for(int i = 0;i < n;i++)
    s++;
```



T(n) = 3n + 2



O(n)

算法时间复杂度 简易判断

- 只有1层循环，循环变量i 每次增加1，或减少1，时间复杂度为O(n)

例： for(int i = 0; i < n; ++i)

- 只有1层循环，循环变量每次乘以2，或除以2，时间复杂度为O(logn)

例：for(int i = 0; i < n; i *= 2)

- 有两层循环，循环变量i,j每次增加1，时间复杂度为$O(n^2)$

例: for(int i = 0; i < n; ++i)

for(int j = i; j < n; ++j)

练习

[J-2015-19]某算法的计算时间表示为递推关系式 $T(n)=T(n-1)+n$为正整数及$T(0)=1$，则该算法的时间复杂度为(　)。

A. O(logn)

B. O(nlogn)

C. O(n)

D. $O(n^2)$

正确答案：D

算法的时间复杂度

求解算法的时间复杂度的具体步骤是：

1. 找出算法中的基本语句。
2. 计算基本语句的执行次数的数量级。
3. 用大O记号表示算法的时间性能。

例题

输入n，输出2^n

请问两种算法的时间复杂度

```
int main()
{
    int n, a = 1;
    cin >> n;
    for(int i = 1; i <= n; ++i)
        a *= 2;
    cout << a;
    return 0;
}
```

快速幂

```
int main()
{
    int n, r = 1, a = 2;
    cin >> n;
    while(n > 0)
    {
        if(n % 2 == 1)
            r *= a;
        a *= a;
        n /= 2;
    }
    cout << r;
    return 0;
}
```

算法的空间复杂度

空间复杂度(Space Complexity)是对一个算法在运行过程中临时占用存储空间大小的量度。 一个算法占用的存储空间记为S(n)，n为数据规模。空间复杂度为O(S(n))。

```
int i = 1;
int j = 2;
++i;
j++;
int m = i + j;
```



S(n) = 3，O(S(n)) = O(1)

```
int[] m = new int[n];
for(int i=0; i<n; ++i)
    m[i] = 2*i;
```



S(n) = 1+n，O(S(n)) = O(n)

时间复杂度分析

当代计算机1s内可做10^7左右次计算

复杂度	数量级	最大规模
O(log n)	>>10^{20}	很大
O(n)	10^6	10^7
O(n log n)	10^5	10^6
O(n^2)	1000	2500
O(n^3)	100	500
O(2^n)	20	20
O(n!)	9	10

空间复杂度分析

内存限制65536KB，请问int型数组可以声明的最大长度

```
int a[?];
```

65536KB = 67108864B

67108864 / 4 = 16777216

数组长度可以为10^6数量级。

各种复杂度的问题

- **输入规模**：一个问题的输入规模是保存输入数据所需要的比特位数。
- **多项式时间算法**：在输入规模为n的情况下，算法能够在$O(n^k)$ 时间(k为常数)内解决该问题。

  例：求最值，冒泡排序
- **伪多项式时间算法**：算法的时间复杂度是输入数据大小的多项式时间表达，但却是输入规模的指数时间表达。

  例：判断素数，背包问题

各种复杂度的问题

- **P问题**（Polynomial），是指能在多项式时间内解决的问题，即解决该问题的算法的时间复杂度为$O(n^k)$。
- **NP问题**（Non-deterministic Polynomial），能在多项式时间复杂度内验证一个解的问题；
- **NPC问题**（NP完全问题 NP-Complete）：所有NP问题在多项式时间内都能约化到它的NP问题。
- **NPH问题**（NP难问题 NP-Hard）：所有NP问题在多项式时间内都能约化到它的问题(不一定是NP问题)。

各种复杂度的问题

P问题：求最短路

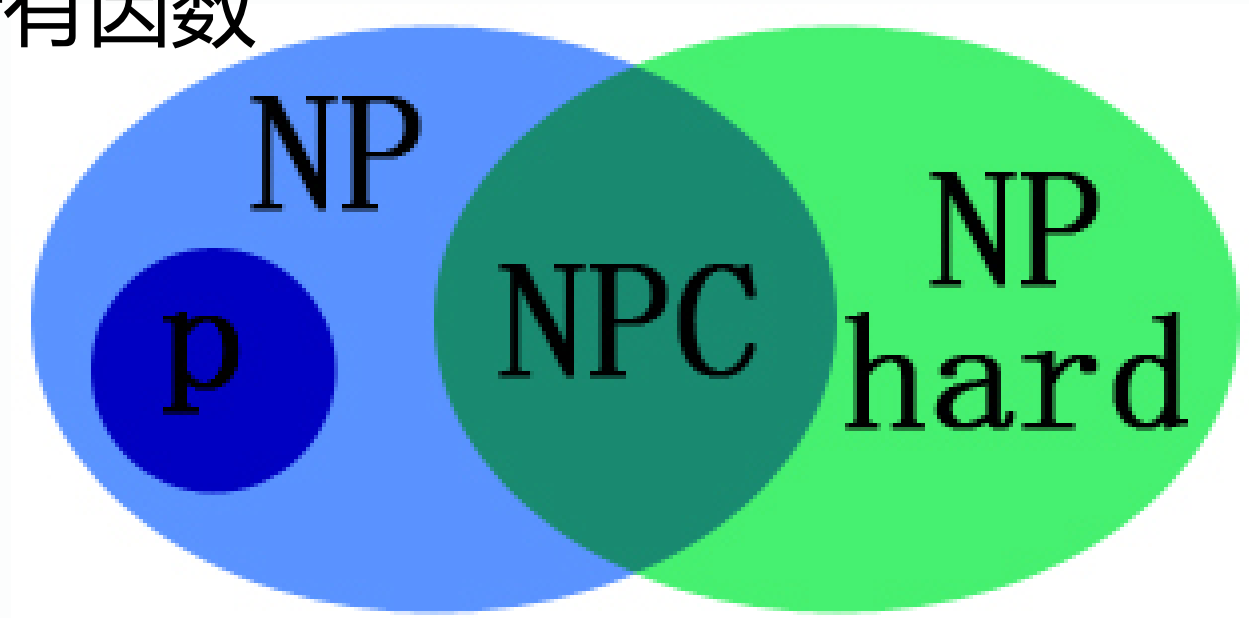
NP问题：找整数n的所有因数

NPC问题：

逻辑电路问题

旅行商问题

背包问题



P=NP问题

P=NP？问题是克雷数学研究所高额悬赏的七个千禧年难题之一，同时也是计算机科学领域的著名难题，该猜想至今未能被证明或证伪。

如果P=NP成立，则所有NP问题（包括NPC问题）都有多项式复杂度的解法。

如果对某个NPC问题找到了多项式时间的解法，那么就证明了所有NP问题都有多项式时间的解法，也就是P=NP

练习

[S-2012-20]（多选）以下关于计算复杂度的说法中，正确的有（ ）。

A. 如果一个问题不存在多项式时间的算法，那它一定是 NP 类问题

B. 如果一个问题不存在多项式时间的算法，那它一定不是 P 类问题

C. 如果一个问题不存在多项式空间的算法，那它一定是 NP 类问题

D. 如果一个问题不存在多项式空间的算法，那它一定不是 P 类问题

正确答案： BD

02 伪代码



伪代码

伪代码是一种介于自然语言和形式化语言之间的半形式化语言，是一种用于描述功能模块的算法设计和加工细节的语言，也称为**程序设计语言。**

描述	伪代码	C++代码
赋值	x←0	x = 0
交换变量	x↔y	swap(x, y)
数组	a[i]	a[i]
	a[1...i]	a[1],a[2],...,a[i]
代码块	缩进表示	用{ }表示

伪代码

描述	伪代码	C++代码
选择结构	if 条件 then 　分支1 else if then 　分支2 endif	if(条件) {分支1} else {分支2}
循环结构	while 条件 do 　循环体 end	while(条件) {循环体}
	for 变量=初值 to 终值 do 　循环体 end	for(变量=初值;变量<=终值;变量变化){循环体}

伪代码练习

【NOIP2020普及组第6题】设A是由实数组成的数组，考虑下面的递归算法：

```
XYZ (A[1..n])
1.  if n= 1 then return A[1]
2.   else temp ← XYZ (A[1...n-1])
3.         if temp < A[n]
4.         then return temp
5.         else return A[n]
```

请问算法XYZ的输出是什么？（B）。

A. A数组的平均
B. A数组的最小值
C. A数组的中值
D. A数组的最大值

03 主定理



主定理

主定理（Master Theorem）适用于求解如下递归式算法的时间复杂度：

$$T(n) = aT(\frac{n}{b}) + f(n)$$

- T(n)是解决规模为n的问题的时间频度
- n 是问题规模大小；
- a 是原问题的子问题个数；
- n/b 是每个子问题的规模大小
- f(n) 是将原问题分解成子问题和将子问题的解合并成原问题的解的时间。

主定理示例

```
int maxVal(int a[], int l, int r)
{
    if(l == r)
        return a[l];
    int m = (l+r)/2;
    return max(maxVal(a, l, m), maxVal(a, m+1, r));
}
```

求a[l]~a[r]的最大值，记n=r-l+1，T(n)=?

T(n) = 2*T(n/2)+3

主定理（非严格描述）

$$T(n) = aT(\frac{n}{b}) + f(n)$$

- 当 $O(n^{\log_b a})$ **大于** O(f(n)) 时：$O(T(n)) = O(n^{\log_b a})$
- 当 $O(n^{\log_b a})$**小于** O(f(n)) 时：$O(T(n)) = O(f(n))$
- 当 $O(n^{\log_b a}\log^k n)$ **等于** O(f(n)) 时：

$$O(T(n)) = O(n^{\log_b a}\log^{k+1} n)$$

特殊地，当k=0时为：

当 $O(n^{\log_b a})$ **等于** O(f(n)) 时：

$$O(T(n)) = O(n^{\log_b a}\log n)$$

练习

例：分治求最值的时间频度递推式为：

T(n) = 2*T(n/2)+3，求该算法的时间复杂度。

a = 2, b = 2, $\log_a b = 1$

$O(n^{\log_a b})$=O(n)

O(f(n))=O(3)=O(1)

O(n) > O(1)

所以O(T(n)) = O(n)

练习

例：归并排序的时间频度递推式为：

T(n) = 2*T(n/2)+n，求该算法的时间复杂度。

a = 2, b = 2, $\log_a b = 1$

$O(n^{\log_a b})$=O(n)

O(f(n))=O(n)

O(n) = O(n)

所以O(T(n)) = O(nlogn)

练习

[S-2017-6]若某算法的计算时间表示为递推关系式:

$T(N)=2T(N/2)+NlogN$

$T(1)=1$

则该算法的时间复杂度为（）

A.O(N)

B.O(NlogN)

C.O(N(logN)^2)

D.O(N^2)

正确答案： C

0 课后作业



作业

OpenJudge NOI 1.3 20:计算2的幂（要求使用快速幂）

OpenJudge NOI 1.6 12:计算2的N次方（高精度快速幂）



谢谢